

CESE5040

High-Performance and AI Architectures

2026 Q4

Student Information

Name: Daniel Tyukov

Student Number: 5714699

Lab: Lab Assignment 3

Notes

- Hardware: same AWS course VM as previous labs (Intel Xeon Platinum 8375C, 4 physical cores / 8 vCPUs, 15 GiB RAM) plus an attached **NVIDIA Tesla T4** (15 GiB, sm_75, CUDA 13.0 driver 580.126.09).
- Software: Python 3.12, NumPy 2.x, CuPy 14.0.1, Numba 0.65.0, JAX 0.10.0, nvprof from the CUDA toolkit, all double precision.
- Each timing is the minimum of 3 runs after one warm-up. GPU timings include a `cuda.synchronize()` both before timer start and at the end so we measure full kernel completion, not just submit.
- Submitted code: `L3_Q1.py`, `L3_Q2.py`, `L3_Q4.py`, `L3_Q5.py`. `tvb_vec_full.py` is the fully refactored NumPy version that `L3_Q1` is built on.

Lab Exercise 3

Question 3.1 — TVB on GPU using CuPy

1, 2, 3. Times for the unmodified `tvb_vec.py`, the fully vectorised NumPy version (`tvb_vec_full.py`), and the CuPy port (`L3_Q1.py`) on all three datasets at `tf=150 ms`:

Implementation	TVB76 (s)	TVB192 (s)	TVB998 (s)
<code>tvb_vec.py</code> (baseline)	9.30	23.81	172.43
<code>tvb_vec_full.py</code> (no per-N loop)	0.41	1.76	57.50
<code>L3_Q1.py</code> (CuPy on Tesla T4)	1.90	1.92	3.04

Table 1: Wall time, min of 3 reps, 150 ms simulation, $dt=0.05$.

4. Discussion. Three things stand out.

The first refactor (`vec_baseline` $\rightarrow$ `vec_full`) is the biggest single jump on the small datasets. Removing the per-centre Python loop turns $N \cdot T$ NumPy calls into T NumPy calls, and the per-call dispatch overhead was where most of the time was sitting at $N = 76$. At TVB998 the speedup shrinks to about $3\times$ because by then the actual $\mathcal{O}(N^2)$ NumPy work dominates the dispatch cost.

For CuPy the surprise is that TVB76 and TVB192 are not faster than plain NumPy — TVB76 is even slower (1.90 s vs 0.41 s). That is the classic small-problem-on-GPU story: the simulation is 3000 timesteps, each one launches around 22 small CuPy kernels (Figure 1 gives the full breakdown), and on a T4 each launch costs roughly 6–10 μ s. Multiply that out and you get ≈ 0.5 s of pure launch overhead before any real arithmetic happens. The GPU can finish the work in well under that.

The picture flips at TVB998. The coupling sum is now an N^2 matmul (998 $\times$ 998) instead of 76 $\times$ 76, and each step has enough work to actually saturate the GPU. CuPy comes in at 3.04 s, beating `vec_full` (57.5 s) by about 19 $\times$, and that gap will widen further at larger N or longer simulations.

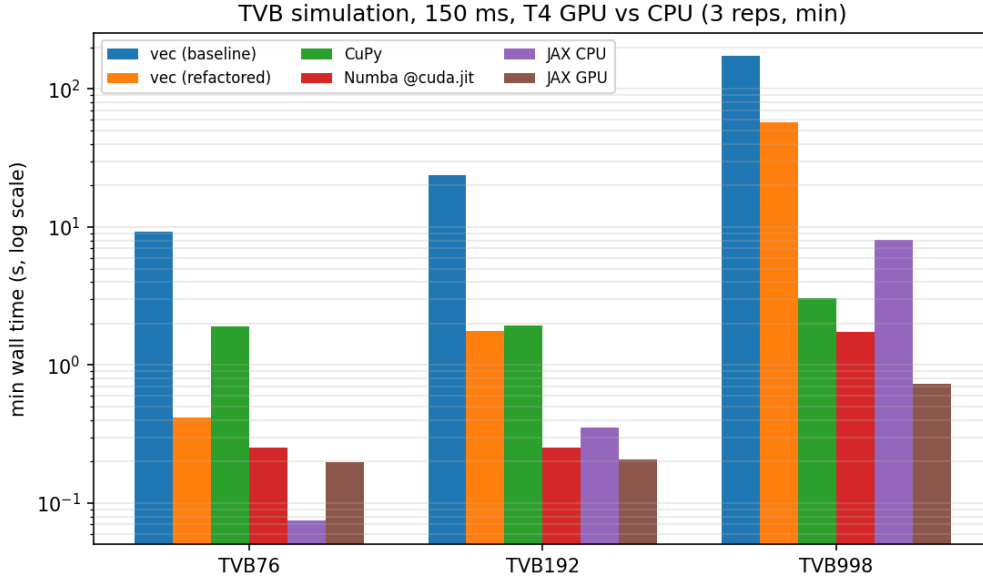


Figure 1: Wall time across all six implementations, log scale. CuPy loses to `vec_full` at TVB76, breaks even at TVB192, and wins by 19 $\times$ at TVB998.

What if we hadn't refactored in step 2? The CuPy version would have been much worse, not better. Each of the N inner iterations would launch its own bundle of CuPy kernels, so for TVB998 that's $998 \cdot 3000 \approx 3$ M kernel launches per run. Even at 6 μ s each that is 18 s of pure launch latency, and the per-launch work shrinks to a single row of the coupling sum, which is also bad for GPU occupancy. Refactoring before porting was the whole point.

5. *nvprof* profile. Nsys 2022.4 on the VM can record `.qdstm` files but its importer is missing, so I used `nvprof` for the textual breakdown instead. Run on TVB192, single rep, no warmup:

The two `dgemm` calls per step (the $[N,2] \cdot [2,64]$ and $[N,64] \cdot [64,2]$ matmuls inside the MLP) dominate at 70.8% of GPU activity. The segmented reduce is the row-sum in the coupling step, the rest is small element-wise kernels. As for data movement: `nvprof` reports a single `[CUDA memcpy HtoD]` category at **101.69 μ s** total over 6 calls, plus a few small `[CUDA memset]`s (8.4 ms). There is no `[CUDA memcpy DtoH]` during the timed loop because we keep `Xs` on the GPU and never copy it back inside `simulate_gpu`. Total GPU activity time was about 966 ms, so data movement is roughly **0.01%** of GPU time and a similarly tiny fraction of the 3.3 s wall. The wall-time gap above the GPU activity time (3.3 s vs 0.97 s) is launch latency from `cuLaunchKernel`, not transfers.

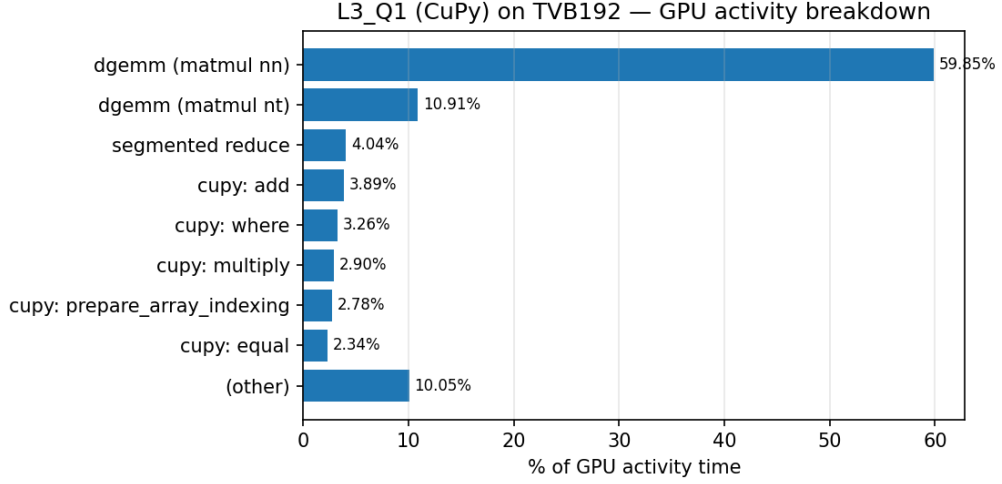


Figure 2: nvprof GPU activity breakdown for L3_Q1 on TVB192, 150 ms. Two cuBLAS dgemm kernels (the MLP matmuls) take 70.8% of all GPU time on their own.

Question 3.2 — High-throughput TVB on GPU (L3_Q2.py)

Implementation. I added a leading K axis to every state and output array. Xs is now $[K, N, M, T]$, the MLP matmul becomes a batched $[K, N, M] @ [M, L]$ (NumPy/CuPy do this automatically), and the coupling indices are computed once per step and broadcast across K . W and D are uploaded to the GPU exactly once and reused for every simulation.

K	Wall (s)	Throughput (iters/s)	Back-to-back L3_Q1 (s)	Throughput (iters/s)
100	7.42	40,412	192.0	1,562
250	18.15	41,314	480.1	1,562
500	36.20	41,439	960.2	1,562

Table 2: TVB192, 150 ms (3000 timesteps per simulation). Back-to-back L3_Q1 numbers are extrapolated as $K \cdot 1.92$ s using the TVB192 single-sim time from Q3.1.

Which is better and why? Batched, by about $26\times$ at every K . The reason is the launch-overhead story from Q3.1, but flipped on its head: when you run L3_Q1 sequentially you pay $T \cdot K \cdot$ (launches per step) kernel launches in total, and at TVB192 each individual kernel only does about 192 nodes of work, which is way below saturation for a T4. Batching keeps the same number of launches ($T \cdot$ launches per step, no K) and just makes each kernel K times bigger, so the SMs actually fill up. The per-step matmul becomes $[K \cdot N, 2] @ [2, 64]$ which is a real piece of work. Throughput rises from 1.56k iters/s to 41k iters/s ($\approx 26\times$) and stays roughly flat from $K = 100$ on, which suggests the T4 is now saturated and the only thing growing with K is the wall time.

Question 3.3 — nvprof of L3_Q2

The same kernels appear, but the proportions shift: dgemm goes from 71% (single sim) to 32%, and the gather (cupy_take) and its surrounding element-wise kernels (multiply, where, subtract) take a much bigger slice. That makes sense — every per-step element-wise op now has $K \cdot N$ elements of work

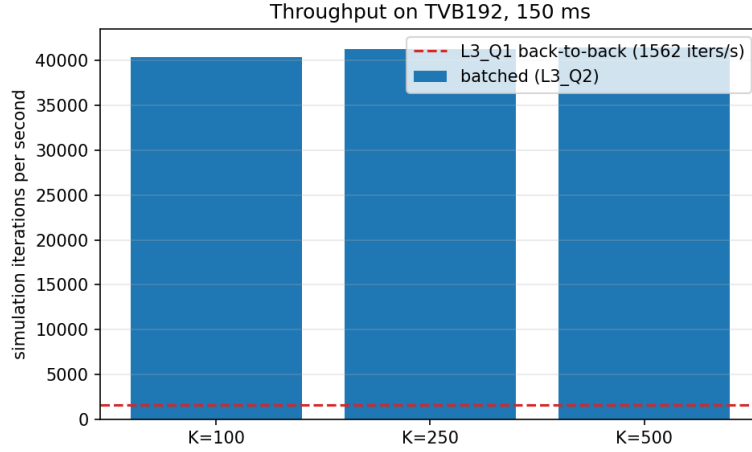


Figure 3: Iterations per second on the same GPU, batched vs running L3_Q1 sequentially K times.

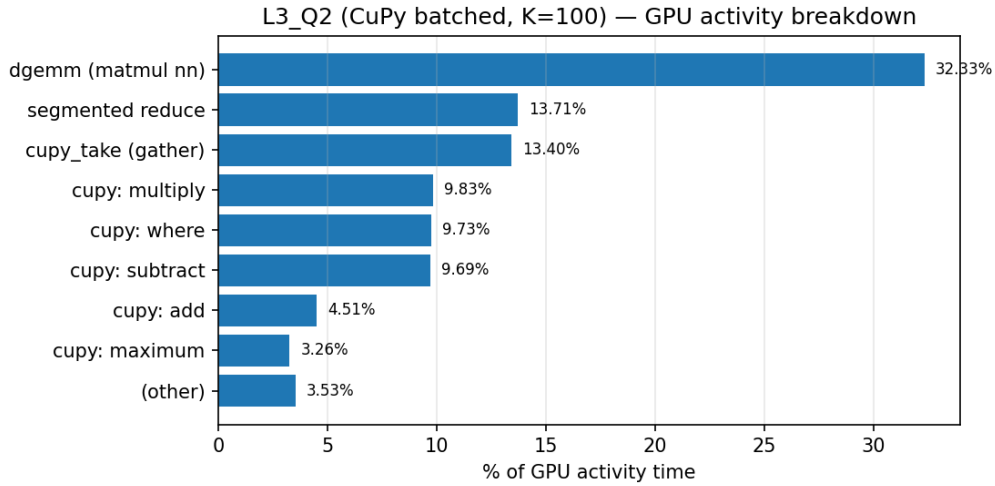


Figure 4: nvprof breakdown of L3_Q2 on TVB192, $K = 100$, 150 ms.

instead of N , so they grow linearly with K , while the dgemm cost grows sub-linearly because the batched dgemm is more efficient.

Data movement: again a single [CUDA memcpy HtoD] block at **101.69 μ s** (the W/D upload at the start), no DtoH inside the timed region, and [CUDA memset] contributes 8.2 ms. Total GPU activity was about 7.5 s, so data movement is around **0.001%** of GPU time. As soon as state lives on the GPU, transfers are not the bottleneck — kernel launches and per-step element-wise work are.

Question 3.4 — TVB on GPU with Numba @cuda.jit (L3_Q4.py)

Implementation. I wrote one @cuda.jit kernel that runs a full timestep. Each thread handles one brain region: it walks all N source nodes to compute the coupling sum into a register, runs the $[2 \rightarrow 64 \rightarrow 2]$ MLP fully unrolled (also in registers), adds the coupling to the second output, and writes the new $[x, y]$ back into $Xs[n, :, t]$. The timestep loop is in Python and just launches the kernel $T - 1$ times. Threads only ever write disjoint slots in Xs so I don't need atomics, shared memory or any extra

synchronisation.

Implementation	TVB76 (s)	TVB192 (s)	TVB998 (s)
L3_Q1 (CuPy)	1.90	1.92	3.04
L3_Q4 (Numba cuda.jit)	0.25	0.25	1.74
speedup vs CuPy	7.5×	7.7×	1.7×

Table 3: Single-simulation runtime, 150 ms.

Numba is faster than CuPy across the board. The main reason is the launch count: CuPy issues about 22 kernels per timestep (one per NumPy-style op), Numba issues exactly one. With $T = 3000$ timesteps and a per-launch cost of $\approx 6 \mu\text{s}$ that alone is around 0.4 s saved. The gap shrinks at TVB998 because there the actual N^2 coupling work inside each kernel becomes the bottleneck rather than the launch overhead, and at that point cuBLAS dgemm (what CuPy uses) is genuinely faster than my hand-written nested loop. Even so Numba is still about $1.7\times$ faster overall because the same launch-count saving applies.

Multi-simulation version. The kernel as written assumes X_s is shape $[N, M, T]$ and uses `1D cuda.grid(1)` indexing. To run K sims at once I would:

- Allocate X_s as $[K, N, M, T]$ (and pre-fill the `[:, :, :, 0] = -1.0` initial condition for all K).
- Switch to a 2D grid: `launch kernel[(N+TPB-1)//TPB, K, TPB, 1]` so that `n = cuda.grid(1)[0]` indexes the brain region and `k = cuda.blockIdx.y` indexes the simulation. Inside the kernel every $X_s[\dots]$ access gains a leading `[k]` index. W and D stay shared across all K .
- Keep the timestep loop in Python so all $K \cdot N$ threads finish step t before $t + 1$ starts (the kernel-launch boundary is the per-timestep barrier).

The W, D matrices are read-only and identical across simulations, so nothing needs to change for them. There is also no inter-simulation coupling so no atomics either.

Question 3.5 — TVB with JAX (L3_Q5.py)

Implementation. The simulation function is the same shape as `L3_Q1.py`'s `simulate_gpu`, but rewritten in JAX functional style. The Python timestep loop becomes a `jax.lax.scan` so that the whole simulation compiles into a single XLA program; `Xs.at[:, :, t].set(new_X)` replaces the in-place write, conditionals on validity become `jnp.where`, and the function is wrapped in `@jax.jit`. The backend is selected via the `JAX_PLATFORMS` env var (`cpu` or `cuda`) before `jax` is imported. `x64` is enabled per the assignment hint.

A few takeaways:

- **JAX CPU is the fastest CPU implementation by a wide margin.** 74 ms on TVB76 vs `vec_full`'s 414 ms ($5.6\times$), 0.35 s vs 1.76 s on TVB192 ($5\times$), 8.03 s vs 57.5 s on TVB998 ($7.2\times$). XLA fuses

Implementation	TVB76 (s)	TVB192 (s)	TVB998 (s)
NumPy (vec_full)	0.41	1.76	57.50
CuPy (L3_Q1)	1.90	1.92	3.04
Numba @cuda.jit (L3_Q4)	0.25	0.25	1.74
JAX (CPU)	0.07	0.35	8.03
JAX (GPU)	0.20	0.21	0.73

Table 4: Single-simulation wall time, 150 ms, min of 3 reps after one warmup. Warmup matters for JAX because the first call triggers the XLA compile.

the entire scan body into one compiled function, so the per-step Python dispatch into many small NumPy ops disappears.

- **JAX GPU is the fastest implementation overall on TVB998 (0.73 s).** It is faster than Numba’s `@cuda.jit` (1.74 s) and far faster than CuPy (3.04 s). On TVB76 it loses slightly to JAX-CPU because the GPU’s launch + small-kernel overhead dominates a 76-node problem, but it is still in the same ballpark as Numba.
- **The same source code runs both backends**, with no code changes — just an env var. That is the practical upside the assignment description was pointing at.

The reason JAX-GPU beats both CuPy and Numba on TVB998 is fusion. XLA can take the whole scan body and emit a small set of fused kernels per step instead of CuPy’s 22 separate ones, and unlike Numba it can use cuBLAS for the heavy matmul instead of an unrolled inner loop. On TVB998 those two effects compound. On TVB76/192 the GPU cost is dominated by launches and the small kernels, so the three GPU backends end up close to each other.

LLM Usage Disclosure

I used Claude Code on this lab for:

- Pair-coding the benchmark harness (`bench_vec.py`, `aggregate.py`, `make_plots.py`) and the runner script.
- Cross-checking the JAX scan / functional-update idioms against the JAX docs while porting from the NumPy version.
- Editing the report (cutting filler, tightening phrasing).

I wrote the GPU kernels (CuPy, Numba `@cuda.jit`, JAX scan body) and ran every benchmark myself on the assigned AWS T4 instance.